異種TEE環境における構成証明代理検証手法の提案と評価

千田 拓矢[1,a)]　佐々木 一也[1]　寺田 雅之[1]

概要：近年，クラウドや分散システムの発展に伴い，異なる組織間で安全に通信・連携を行うためのリモートアテステーションの重要性が高まっている．一方，機密コンピューティングの普及により AWS Nitro Enclaves や AMD SEV-SNP など多様な TEE が併存し，組織ごとに異なる TEE やクラウド環境を採用することが想定される．構成証明レポートの仕様はベンダごとに異なるため，異種 TEE 間ではサーバごとに個別の検証ロジックが必要となり，開発・保守コストの増大が課題となる．本研究では，階層的リモートアテステーションにおいて，リーダーが構成証明レポート本体を変更せずに TEE 種別等を含む共通ヘッダを付加してカプセル化し，検証プロトコルを動的に選択する手法を提案する．新たな TEE を追加する際の変更をリーダーのみに限定し，検証ロジックの開発・保守コストを $O(M \times N)$ から $O(M + N)$ に低減する．実機評価では，Nitro Enclaves と AMD SEV-SNP の 3 通りの組み合わせすべてで相互アテステーションが成立し，共通ヘッダの付加・検証に伴うオーバーヘッドは処理全体の 0.5 %未満にとどまることを示した．

キーワード：クラウド，機密コンピューティング，リモートアテステーション，TEE

Design and Evaluation of a Proxy Attestation Verification Method for Heterogeneous TEE Environments

Takuya Chida[1,a)]　Kazuya Sasaki[1]　Masayuki Terada[1]

Abstract: With the spread of confidential computing, diverse TEE technologies such as AWS Nitro Enclaves and AMD SEV-SNP coexist, and organizations are expected to adopt different TEEs. Since attestation report formats differ across vendors, remote attestation between organizations adopting heterogeneous TEEs requires per-server verification logic, increasing development and maintenance cost. We propose a hierarchical remote attestation method in which a leader encapsulates each attestation report, leaving it intact, with a common header carrying the TEE type, and dynamically selects the verification protocol accordingly. Confining the changes needed to support a new TEE to the leader alone reduces the development and maintenance cost of verification logic from $O(M \times N)$ to $O(M + N)$. Evaluation on real AWS instances showed that mutual attestation succeeded in all three combinations of Nitro Enclaves and AMD SEV-SNP, and that the overhead of the common header remained below 0.5 % of total processing time.

Keywords: Cloud Computing, Confidential Computing, Remote Attestation, TEE

[1] 株式会社 NTT ドコモ モバイルイノベーションテック部
Mobile Innovation Tech Department, NTT DOCOMO, INC.
[a)] takuya.chida.mn@nttdocomo.com

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

1.1 背景

クラウドおよび分散システムの発展に伴い，異なる組織が保有するデータを相互に連携させて活用する需要が増大している．このような組織間データ連携においては，相手組織のシステムが正当なソフトウェアによって構成され意図した通りに動作していることを技術的に確認する手段が不可欠である．例えば，医療や金融のように機微なデータを扱う分野では，複数の組織がデータを持ち寄って共同で分析する際，データを預ける相手環境の正当性の確認が前提となる．Trusted Execution Environment（TEE）はハードウェアベースの分離実行環境を提供することでこの要求に応える基盤技術であり，TEE 内で稼働するソフトウェアの完全性を遠隔から検証する手法をリモートアテステーション（構成証明）と呼ぶ．

著者らはこれまで，大規模分散システムにおけるリモートアテステーションの効率化を目的として，リーダー／フォロワー構成による階層的リモートアテステーション手法を提案してきた [9]．同手法では，各組織内のリーダーがフォロワーの構成証明を集約して検証し，組織間ではリーダー同士のみが相互に構成証明を行う．

1.2 課題

前回研究では，対象とする TEE を AWS Nitro Enclaves のみに限定していた．しかし実際には，機密コンピューティングの普及に伴い AWS Nitro Enclaves，AMD SEV-SNP，Intel TDX など多様な TEE が併存しており，組織ごとに採用する TEE 環境が異なることが想定される．構成証明レポートの仕様は，データ形式（CBOR ／固定長バイナリ構造体／ SGX Quote 互換構造），測定値の表現（PCR ／ 48 byte ダイジェスト／複数の MR レジスタ），および信頼の起点（クラウドベンダの PKI ／ハードウェアベンダの鍵配布サービス）のいずれにおいてもハードウェア・クラウドベンダごとに大きく異なる．

このため，異種 TEE を採用する組織間でリモートアテステーションを行う際には，以下の課題が生じる．

検証ロジックの実装コスト（$M \times N$ 問題） 検証を行うすべてのサーバが，対向が採用しうるすべての TEE 種別の検証ロジックを個別に保持する必要がある．システム全体で扱う TEE 種別数を M，サーバ台数を N とすると，実装箇所は $M \times N$ に比例して増大する．

拡張時の保守コスト 新たな TEE を 1 種類追加するたびに既存の全サーバの実装を更新して再デプロイする必要があり，保守性・拡張性に乏しい．

実装差異に起因するセキュリティリスク サーバごとに検証ロジックを実装することは，実装漏れや検証手順の誤りが局所的に生じる余地を残す．

1.3 本研究の提案

本研究では，階層的リモートアテステーションのリーダーサーバが，構成証明レポート本体を変更せずに TEE 種別（`tee_type`）等を含む**共通ヘッダ**を付加してカプセル化する手法を提案する．リーダーは共通ヘッダの `tee_type` に基づいて適切な検証プロトコルを動的に選択して代理検証を行い，検証に成功した場合にのみ自身の鍵で共通ヘッダに署名（代理署名）する．受信側は，ネイティブな構成証明レポートを自ら再検証する必要はなく，リーダーの署名のみを検証すればよい．

この構成により，検証ロジックの抽象化がリーダー側に集約される．新たな TEE を追加する際に変更が必要な箇所はリーダーの検証プロトコル対応表のみに限定され，開発・保守コストは $O(M \times N)$ から $O(M + N)$ に低減される．例えば TEE 2 種・サーバ 100 台の環境では，従来方式の実装箇所は 200 箇所に達し，TEE を 1 種類追加すると 100 台すべての改修を要するのに対し，提案方式ではリーダーのみの変更で済む．

1.4 本稿の貢献

本稿の貢献は以下の通りである．

- 構成証明レポートを不変のまま保持しつつ検証プロトコルの動的選択を可能とする共通ヘッダ方式を提案した．
- AWS Nitro Enclaves および AMD SEV-SNP を対象に提案手法を実装し，AWS 上の実機を用いた 3 通りの組み合わせの相互アテステーションにより異種 TEE 間の相互運用性を実証した．
- 提案手法が追加する処理のオーバーヘッドが処理全体の 0.5 %未満であること，およびフォロワーの台数（1 台から 2 台）や TEE 種別の組み合わせを変化させても 1 件あたりの処理時間が劣化しないことを実測により示した．
- 検証ロジックの開発・保守コストを実装規模に基づき定量比較し，従来方式の $O(M \times N)$ に対し提案方式が $O(M + N)$ となることを示した．

1.5 本稿の構成

本稿の構成は以下の通りである．まず第 2 章で関連研究を整理し，本研究の位置づけを明確にする．第 3 章では，共通ヘッダによるカプセル化と検証プロトコルの動的選択からなる提案手法を述べ，第 4 章ではその実装と実験環境を述べる．第 5 章では，提案手法の処理オーバーヘッドと異種 TEE 構成に対する処理時間の一貫性を評価する．第 6 章では，セキュリティ強度や拡張性等の観点から考察を行う．最後に，第 7 章で結論と今後の課題を述べる．

© 2026 Information Processing Society of Japan

2. 関連研究

RFC 9334[1] はリモートアテステーションの基本アーキテクチャを標準化しており，本研究のリーダーは同アーキテクチャの Verifier と Relying Party を仲介する位置づけにある．

同種のデバイス群を対象に通信量の削減を図る研究群として SEDA[2]，Kwon の手法 [7]，SCRAPS[6] があり，Intel SGX を対象とした相互アテステーションの研究として OPERA[3]，DMA[5] があるが，いずれも単一種の TEE を前提としており，異種 TEE のレポート仕様の差異は扱っていない．

これに対し，異種 TEE 間のアテステーションを直接扱った研究として Chen らの MAGE[4] が挙げられる．MAGE は信頼できる第三者を介さずにエンクレーブ群が相互にアテステーションを行う仕組みにより，複数種の TEE をまたぐ信頼確立を可能とした．ただしグループ内の各エンクレーブが相互に相手の測定値を検証する分散型の構成であり，各エンクレーブが対向の TEE 種別に応じた検証能力を保持することを前提とする．

構成証明を通信チャネルと結合する手法として，Knauth らの RA-TLS[8] は TLS の X.509 証明書拡張領域に構成証明レポートを直接埋め込む方式を提案している．接続の確立と信頼の確立を原子的に行える利点がある一方，埋め込まれたレポートの検証は受信側の責務のまま残るため，レポート形式ごとに個別の検証ロジックを要する点は従来方式と変わらない．また，TLS ライブラリ側の改修が必要であり，Nitro Enclaves のように証明書チェーンを内包する巨大なレポートは扱いにくい．

一方，著者らの前回研究 [9] は，第 1 章で述べたリーダー／フォロワーの階層構成により，全サーバが総当たりで相互検証する方式と比べてスケーラビリティに対応した手法である．

以上のように研究は蓄積されてきたが，「異種 TEE のレポート仕様の差異を吸収する抽象化層を階層構成のどこに置くか」という設計上の観点，およびそれに伴う開発・保守コストを明示的な評価軸とした検討は十分ではない．本研究は，前回研究の階層構成を土台としつつ，リーダーを異種 TEE の抽象化点として位置づけることで，実運用における拡張コストの低減を図るものである．

3. 提案手法

前回研究 [9] のリーダー／フォロワーによる階層構成は，単一種の TEE を前提としていた．本研究では，この階層構成を土台としつつ，異種 TEE が混在する環境に対応するため，構成証明レポートを共通ヘッダによってカプセル化し，検証プロトコルを動的に選択する仕組みを新たに導入する．

表 1 共通ヘッダのフィールド構成

フィールド	役割
`version`	ヘッダ形式のバージョン
`tee_type`	TEE 種別．検証プロトコル選択のキー
`src`	ヘッダ発行元（リーダー）の識別子
`timestamp`	ヘッダ生成時刻．有効期限判定に使用
`nonce`	検証側が発行した nonce．リプレイ対策
`verification_meta`	用いた検証プロトコルと測定値
`native_report`	TEE 固有の構成証明レポート本体（不変のまま格納する）
`signature`	上記全体に対するリーダーの署名

3.1 システムモデルと前提条件

本研究は，各組織がそれぞれ独立にクラウド上にシステムを構築し，組織間で安全にデータ連携を行う環境を前提とする．提案手法の全体構成を**図 1** に示す．各組織は自組織のシステムを信頼できるものと仮定するが，相手組織のシステムは信頼しない．各組織内では，リーダー（Leader）とフォロワー（Follower）の 2 種の役割を設ける．

リーダーは組織を代表するサーバであり，同一組織内のフォロワーの構成証明を検証（インターナルアテステーション）するとともに，対向組織のリーダーとの間で相互に構成証明を行う（グローバルアテステーション）．リーダー自身も TEE 上で動作し，構成証明の対象となる．フォロワーはアプリケーション処理を担うサーバであり，自身の構成証明レポートをリーダーに提出し，リーダーからクレデンシャル情報を受領する．

本研究では，各組織（およびその配下のサーバ）が互いに異なる TEE やクラウド環境を採用しうることを想定する．すなわち，グローバルアテステーションを行う 2 台のリーダーの TEE 種別が一致しない場合，および 1 台のリーダーが複数の TEE 種別のフォロワーを配下に持つ場合の双方を対象とする．なお，構成証明に必要なルート証明書・期待される測定値・対向リーダーの署名検証用公開鍵は，事前に安全な手段で取得済みであることを前提とする．特にプログラムのハッシュ値などの測定値は，なりすましを防ぐため対向の自己申告による共有はしないものとする．

3.2 共通ヘッダの設計

提案手法では，TEE 固有の構成証明レポートを共通ヘッダによってカプセル化する．**表 1** に共通ヘッダのフィールド構成と各フィールドの役割を示す．

本設計における要点は以下の 3 点である．

第一に，構成証明レポート本体（`native_report`）を一切変更せずそのまま格納し，ヘッダのみを付加する．これにより，レポート本体に含まれるベンダの署名の真正性を損なうことなく検証ロジックを特定するための情報を付与でき，受信側がレポート本体を独自に再検証する余地も残

© 2026 Information Processing Society of Japan

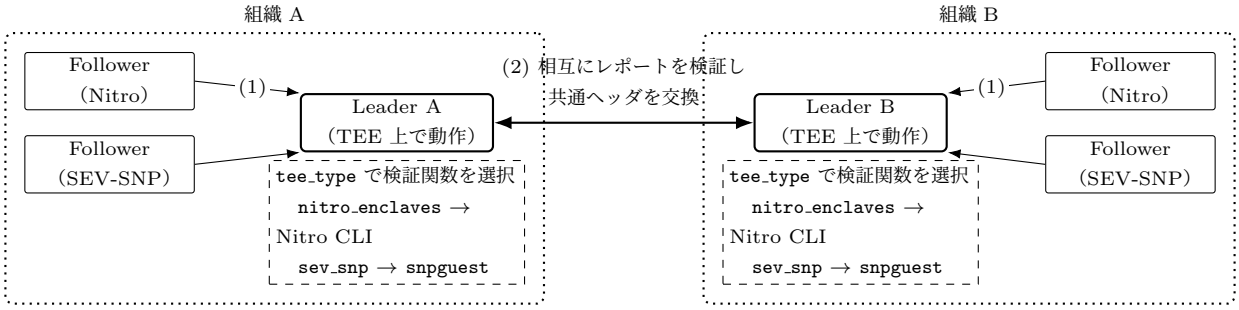


図 1 提案手法の全体構成．(1) インターナルアテステーション: リーダーがフォロワーの構成証明レポートを検証し，共通ヘッダを付加する．(2) グローバルアテステーション: リーダー同士が相互に構成証明レポートを検証し，署名付き共通ヘッダを交換する

される．

第二に，`signature` は共通ヘッダの生成主体であるリーダー自身の鍵による署名であり，「リーダーが代理で検証した」という事実に対する保証（代理署名）として機能する．受信側はリーダーの署名のみを確認すればよく，対向の TEE 種別に応じた検証ロジックを保持する必要がない．

第三に，検証に失敗した場合は共通ヘッダを生成しない設計とする．検証結果を状態フラグとして保持する設計も考えられるが，ヘッダの存在そのものが検証成功を意味するようにすることで，受信側が状態フラグの解釈を誤って不正なレポートを受理する危険を構造的に排除できる．

リプレイ攻撃への対策としては，検証側が発行した `nonce` を共通ヘッダと，各 TEE のネイティブレポートが備える任意データ領域（Nitro Enclaves の `nonce` フィールド，AMD SEV-SNP の REPORT_DATA）の双方に紐付ける設計とし，`timestamp` によって有効期限を判定する．照合を TEE ごとの検証プロトコルの内部に閉じ込めることで，埋め込み方式の差異を吸収しつつ上位層では `nonce` と `timestamp` のみを扱えばよい．

なお，共通ヘッダの表現形式には，可読性と柔軟性を優先して JSON を用いる．

3.3 リーダーによる検証プロトコルの動的選択

リーダーは，共通ヘッダの `tee_type` をキーとするディスパッチテーブル（`tee_type` から検証関数への対応表）によって検証ロジックを選択する．すべての検証関数は `verify(native_report, **kwargs)` という共通のシグネチャに揃えられており，TEE 固有の差異は各検証関数の内部に閉じ込められる．`verify()` はネイティブレポート本体の正当性（署名・測定値）に加え，`kwargs` として渡される `nonce` の一致と `timestamp` の有効期限も確認し，すべてを満たす場合にのみ成功を返す．この照合ロジックも各 TEE の検証関数の内部に閉じ込められるため，リプレイ攻撃対策は上位層に影響を与えることなく検証プロトコル内で完結する．また，検証関数はフォロワーごとの状態を保持しないステートレスな設計であり，1 件の検証に要する時間はフォロワーの台数に依存しない．

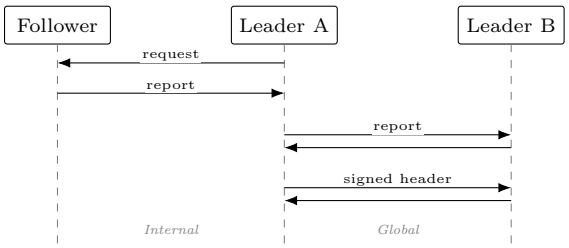


図 2 提案手法における処理フロー．検証は各レポート受信直後にリーダー側で行われ，成功時のみ次の矢印に進む

新たな TEE 種別を追加する際に必要な変更は，(1) 当該 TEE 用の検証関数の追加と，(2) ディスパッチテーブルへの 1 エントリの登録のみである．既存の検証関数，共通ヘッダの生成・検証ロジック，および既存のフォロワー側の実装には変更が生じない．

3.4 処理フロー

提案手法における処理フローを **図 2** に示す．インターナルアテステーションとグローバルアテステーションのいずれにおいても，検証側は同一の共通ヘッダ生成処理を用いる．グローバルアテステーションの完了時点で双方のリーダーが「相手リーダーの共通ヘッダ（署名付き）」を保持した状態となり，以降のリーダー間・リーダー／フォロワー間のクレデンシャル情報の配布は共通ヘッダの署名検証のみで済む．

グローバルアテステーションでは，双方のリーダーが相手リーダー自身のネイティブ構成証明レポートを検証する．これにより，リーダー自身が TEE 上で正当に動作していることが対向組織から確認され，そのリーダーが発行する代理署名（共通ヘッダへの署名）も信頼できるものとなる．すなわち，リーダー間の相互アテステーションは，リーダー自身の構成証明にとどまらず，インターナルアテステーションで検証済みの配下フォロワー全体を含む，組織全体の構成証明を実質的に達成する．

© 2026 Information Processing Society of Japan

3.5 従来手法との比較

検証ロジックの実装箇所数について，従来手法と提案手法を比較する．システム全体で扱う TEE 種別数を M，サーバ台数を N，組織数（したがってリーダー台数）を K とする．

3.5.1 従来手法

従来手法では，すべてのサーバが対向の全 TEE 種別の検証ロジックを保持するため，実装箇所は $M \times N$ となり，新たな TEE を 1 種類追加する際には N 箇所の変更を要する．

3.5.2 提案手法

検証ロジックはリーダーに集約されるため，リーダーが保持する検証ロジックは `tee_type` ごとに 1 つ，すなわち M 個である．一方フォロワーは自身の TEE 向けのレポート生成処理のみを持ち，これはサーバごとに 1 つ，全体で N 個である．したがって実装箇所は $M \times K + N$ となり，実運用において $K \ll N$（1 組織あたり 1 台のリーダー）が成り立つ範囲で $O(M+N)$ と扱える．さらに，リーダー側の検証ロジックを共通ライブラリとして配布できる場合，開発コストとしては M 回の実装で足り，新たな TEE の追加に要する変更は $O(1)$ となる．なお，異種 TEE 間のアテステーションを分散型で実現する手法のように，各ノードが対向の TEE 種別に応じた検証能力を個別に保持する構成では，実装箇所は従来手法と同様に $M \times N$ となり，本コスト軸は改善されない．

4. 実装

本章では，AWS Nitro Enclaves および AMD SEV-SNP を対象とした提案手法の実装を述べる．まず両 TEE の構成証明レポート仕様がどのように異なるかを整理したうえで，この差異を検証アダプタ内に閉じ込める実装の構成と新 TEE 追加時の変更コストの実証を示し，最後に次章の評価に用いる AWS 上の実験環境を述べる．

4.1 対象 TEE と構成証明レポート仕様の差異

本研究では，AWS 上で利用可能な AWS Nitro Enclaves[10] および AMD SEV-SNP[11] の 2 種の TEE を対象とする．両者における構成証明レポート仕様の差異は大きい．Nitro Enclaves の構成証明ドキュメントは CBOR/COSE 形式で PCR 値を測定値として保持し，証明書チェーンをレポート内に同梱するため Nitro CLI/NSM API のみで検証できる．AMD SEV-SNP の構成証明レポートは固定長バイナリ構造体で MEASUREMENT(48 byte) を保持し，AMD KDS から証明書チェーン (ARK/ASK/VCEK) を外部取得したうえで `snpguest` により検証する（信頼の起点となるルート証明書 ARK は第 3.1 節の前提により事前取得済みであり，ここで取得するのは ASK と VCEK のみである）．このように形式・信頼の起点・検証手段のすべてが異なり，検証ロジックを共通化できない．提案手法はこの差異を各検証関数の内部に閉じ込め，上位層には `tee_type` という単一のキーとして露出させる．

表 2 新 TEE を 1 種類追加する際の変更コスト

	従来方式	提案方式
検証ロジックの実装箇所	$M \times N$	$M \times K + N$
変更が必要なサーバ数	N	K
変更ファイル数	N	2
うち既存ファイルの変更	N	1

4.2 実装の構成と変更コストの実証

提案手法を Python 3.12.3 で実装した．実装は TEE 非依存の共通部（共通ヘッダの生成・署名・検証，およびディスパッチテーブル）と，TEE 依存の検証アダプタ部に明確に分離されている[*1]．

共通ヘッダの署名には Ed25519 を用い，署名対象バイト列は JSON のキー順・区切り文字を固定して正規化（canonical 化）することで，フィールドの並び替えによる署名検証の失敗を防いでいる．

Nitro Enclaves 用の検証ロジックは前回研究の実装を移植した．AMD SEV-SNP 用の検証アダプタは，AMD KDS からの証明書チェーン取得と `snpguest` ／ `openssl` の呼び出しをラップしており，証明書チェーンの取得手段を差し替え可能な構造とすることで実機なしで単体試験を実施できるようにしている．

本実装に基づき，第 3.5 節で示したコストの見積りを確認した結果を**表 2** に示す．新たな TEE 種別を追加する際に必要な変更は，当該 TEE 用の検証アダプタの新規追加とディスパッチテーブルの一部の変更に限られる．

4.3 実験環境

異種 TEE 間の相互アテステーションを確認するため，AWS クラウド上に 2 つのアカウント（以下，組織 A および組織 B）を用意し，各環境に Nitro Enclaves 対応インスタンスと AMD SEV-SNP 対応インスタンスをそれぞれ配置する．実験環境を**表 3** に示す．組織 A と組織 B の構成はすべて一致している．両ホストで CPU が同一であり，OS・カーネル・Python のバージョンも一致しているため，第 5 章の処理時間の差はハードウェア性能差ではなく TEE の機構と検証手段の差に起因すると言える．

構成証明レポートおよび共通ヘッダの交換には，組織 A・組織 B の双方からアクセス可能な共有 S3 バケットを用いる．検証に用いる期待値（Nitro Enclaves の PCR0，AMD SEV-SNP の MEASUREMENT）は，第 3.1 節の前提に従い，対向の自己申告を用いず事前に安全な手段で共有した値をローカル設定ファイルから読み込む．

[*1] 実装規模は，共通部が計 216 行，検証アダプタ部が Nitro Enclaves 282 行，AMD SEV-SNP 111 行である．

© 2026 Information Processing Society of Japan

表 3　実験環境

	Nitro Enclaves	AMD SEV-SNP
インスタンス	m6a.xlarge	c6a.xlarge
検証ツール	Nitro CLI 1.4.5	`snpguest` 0.10.0
測定値	PCR0	MEASUREMENT
共通	CPU: AMD EPYC 7R13	
	OS: Ubuntu 24.04.4 LTS	
	言語: Python 3.12.3，リージョン: us-east-2	

5. 評価

本章では，提案手法を AWS 上の実機で評価した結果を示す．評価方法を述べたうえで，共通ヘッダの付加・検証が処理全体に与えるオーバーヘッド（評価 A）と，TEE 種別の組み合わせやフォロワー台数を変化させた際の処理時間の一貫性（評価 B）を実測により明らかにする．

5.1 評価方法

以下の 2 つの観点から提案手法を評価する．なお，新 TEE 追加時の変更コストについては第 3.5 節で理論的な議論を，第 4.2 節で実装に基づく裏付けを示したとおりであり，本章では扱わない．また，本稿の実装ではリーダー側での nonce 照合が未実装であり，評価 A・B で測定する Verify の時間にはこれを含まない．

評価 A（処理オーバーヘッド）　共通ヘッダの付加および検証プロトコルの選択に伴う処理時間を測定する．構成証明レポートの生成（Generate），アップロード（Upload），ダウンロード（Download），検証（Verify）の内訳に加え，共通ヘッダの生成・署名（Encapsulate）および署名検証（Header Verify）を測定し，提案手法の導入によって追加される処理の割合を明らかにする．

評価 B（異種 TEE 構成に対する処理時間の一貫性）　1 台のリーダーが複数のフォロワーを検証する構成において，フォロワーの台数や TEE 種別の組み合わせが 1 件あたりの処理時間を変化させないかを確認する．リーダー 1 台とフォロワー 2 台からなる構成で，フォロワーごとの検証（Verify）とカプセル化（Encapsulate）に要する時間を測定し，評価 A の 1 対 1 構成の実測値から外挿した値と比較する．本評価はインターナルアテステーションのみを対象とし，グローバルアテステーションは含まない．また，台数に関する結論は本評価で実測した 1 台から 2 台に増やした範囲に限る．

5.2 処理オーバーヘッド（評価 A）

以下の測定はいずれも，Nitro Enclaves–Nitro Enclaves，Nitro Enclaves–AMD SEV-SNP，AMD SEV-SNP–AMD SEV-SNP の 3 通りの組み合わせすべてで，双方が対向の発行した共通ヘッダの署名検証に成功し相互アテステーションが成立したうえで取得したものである．とりわけ Nitro Enclaves–AMD SEV-SNP の組み合わせが成立したことは，構成証明レポートの形式・信頼の起点・検証手段のすべてが異なる TEE 間であっても，共通ヘッダによる抽象化を介して相互に信頼を確立できることを示している．

評価 A の結果を**表 4** に示す．各構成について 5 回測定し，平均と標準偏差を示した．アテステーション処理全体の所要時間は実行単位で 76–311 ms であった（以下，範囲は実行単位の最小値と最大値を表す）．S3 を介した通信（Upload，Download）の標準偏差が相対的に大きい一方，検証および共通ヘッダの処理はいずれも標準偏差が小さく，以下に述べる結果は安定して再現される．

第一に，提案手法が追加する処理のコストは無視できる水準である．共通ヘッダの生成・署名（Encapsulate）と署名検証（Header Verify）の合計は 0.26–1.13 ms（平均 0.52 ms）であり，処理全体に占める割合は実行単位で 0.13–0.49 %（平均 0.30 %）にすぎない．この比率の分母は S3 を介した通信（Upload，Download）を含むが，絶対時間も最大 1.13 ms にとどまる．これは，受信側がネイティブレポートを再検証せずリーダーの署名のみを検証すれば足りるという提案手法の設計が，実行時コストの面でも妥当であることを示している．

第二に，検証（Verify）の所要時間は自身の TEE 種別ではなく対向の TEE 種別によって決まる．対向が Nitro Enclaves の場合は 22.3–31.4 ms（平均 25.5 ms）である一方，対向が AMD SEV-SNP の場合は 173.0–186.9 ms（平均 180.6 ms）と約 7.1 倍を要した．これは，AMD SEV-SNP の検証が AMD KDS からの証明書チェーンの外部取得（ネットワーク通信）と `snpguest`・`openssl` のサブプロセス起動を伴うのに対し，Nitro Enclaves は証明書チェーンをレポート内に同梱するため外部通信を要しないという構造的差異に対応している．

第三に，レポート生成（Generate）についても AMD SEV-SNP が Nitro Enclaves の約 10 倍（Nitro Enclaves の 5.5–6.1 ms に対し 56.1–60.1 ms）を要した．これは `snpguest` のサブプロセス起動コストによるものであり，vsock 経由で常時稼働の Enclave に要求を送る Nitro Enclaves の方が軽い．

5.3 異種 TEE 構成に対する処理時間の一貫性（評価 B）

リーダー 1 台にフォロワー 2 台を接続した構成で，リーダープロセス内で最初に検証されたフォロワーの 1 件あたり処理時間を，評価 A の 1 対 1 構成から外挿した値と比較した（各条件 $n = 5$）．結果を**図 3** に示す．

リーダー／フォロワーの 4 通りの TEE 組み合わせいずれも外挿値との差は 5 %以内であった．リーダーはフォロワーごとの状態を保持しないため，これは設計から期待される結果であり，本評価で確認したフォロワー 1 台から 2

© 2026 Information Processing Society of Japan

表 4 評価 A: アテステーション処理時間の内訳（単位: ms，平均 ± 標準偏差，各構成 $n = 5$）

組み合わせ	役割	自 TEE	対向 TEE	Generate	Upload	Download	Verify	Encap.	Hdr.Verify	合計
nitro-nitro	組織 A	Nitro	Nitro	5.7±0.2	29.1±3.5	27.2±2.3	22.7±0.2	0.12±0.00	0.21±0.05	85.0±1.6
	組織 B	Nitro	Nitro	5.8±0.2	26.9±3.9	28.7±7.2	22.9±0.4	0.11±0.00	0.17±0.03	84.6±6.9
nitro-sevsnp	組織 A	Nitro	SEV-SNP	5.8±0.1	27.3±2.0	28.9±4.9	173.2±0.3	0.26±0.01	0.21±0.03	235.6±6.1
	組織 B	SEV-SNP	Nitro	58.2±1.4	33.9±14.2	34.7±9.0	30.8±0.5	0.23±0.13	0.33±0.20	158.2±23.8
sevsnp-sevsnp	組織 A	SEV-SNP	SEV-SNP	58.1±1.1	29.6±4.4	26.8±2.6	184.3±1.7	0.57±0.19	0.29±0.17	299.6±8.0
	組織 B	SEV-SNP	SEV-SNP	58.4±1.2	31.1±1.5	30.4±2.4	184.1±0.7	0.43±0.18	0.23±0.12	304.8±1.4

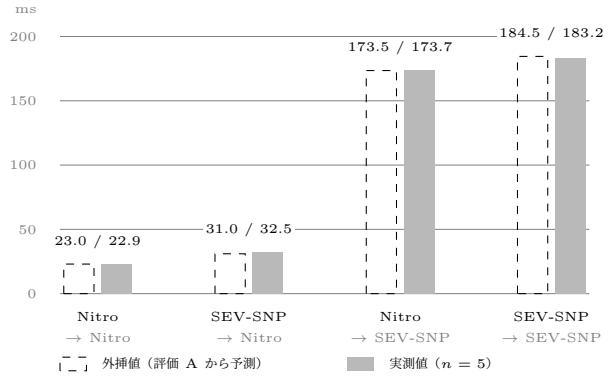


図 3 評価 B: 異種 TEE 構成における外挿値と実測値の比較（単位: ms）．リーダー → フォロワーの 4 通りいずれも外挿値と実測値がほぼ一致する

台の範囲では，台数や TEE 種別の組み合わせによって 1 件あたりの処理時間が変化しないことを実機で確認できた．

なお同一リーダープロセス内で 2 番目以降に検証されたフォロワーでは，同種（Nitro Enclaves）の構成に限り処理時間が 1 番目より大幅に短くなる系統的な現象が観測された．これは提案手法の性質によるものではなく，実装が用いるライブラリの初期化に起因する副次的な効果であるため，第 6.3 節で改めて述べる．外挿値は 1 対 1 構成すなわちプロセス内 1 回のみの実行から得た値であり，2 番目以降とは直接比較できない．

6. 考察

本章では，検証ロジックをリーダーに集約することがセキュリティ強度に与える影響を論じ，次いで新たな TEE やクラウド環境への拡張性を検討する．最後に，評価 B で観測したリーダープロセス内の初期化コストの現象について，その原因と評価結果への影響を述べる．

6.1 セキュリティ強度

検証ロジックをリーダーに集約することで，サーバごとの実装差異に起因する検証漏れや手順の誤りの余地は排除される．一方でこの集約には，(1) リーダーに信頼が集中することに伴うリスクと，(2) 共通ヘッダの改ざん・偽装のリスクがある．

まず，リーダーの署名鍵が漏洩した場合，実際には検証されていない構成を正当なものとして証明できてしまう．このリスクに対しては，リーダー自身も TEE 上で動作し，グローバルアテステーションによって完全性が対向組織から検証されているため，署名鍵を TEE 内で生成し TEE 外に持ち出さない運用と組み合わせることで，実用上許容できる水準に抑えられると考えられる．また，リーダーの TEE に対するゼロデイ攻撃が成立すれば配下のフォロワー全体の構成証明の信頼性が一括して損なわれ，サービス停止攻撃を受ければ組織全体のアテステーションが停止するというリスクも想定される．このように，リーダーは処理負荷と可用性の面での単一障害点であると同時に，侵害されれば組織全体の構成証明の信頼が崩れる単一の信頼点ともなる．これは信頼集約という設計自体に内在するトレードオフであり，リーダーの TEE に対するパッチ適用や再起動の運用を含め，リーダーの信頼性を維持するための運用上の対策が必要である．

共通ヘッダの改ざん・偽装は，`tee_type` を含む全フィールドを署名対象とすることで検知できる．`tee_type` が保護されない場合，攻撃者がこれを書き換えて受信側に誤った検証プロトコルを選択させる攻撃が成立しうるが，正規化したヘッダ全体を署名対象とする本実装ではこの攻撃は成立しない．なお，共通ヘッダの再送によるリプレイ攻撃に対しては，第 3 章で述べたとおり，検証側が発行した nonce をヘッダとネイティブレポートの双方に紐付けることで対処する．nonce は要求ごとに検証側が新たに発行するため，レポートが当該要求に応じて新たに生成されたものであることが保証される．

6.2 拡張性

提案手法は，レポートの形式や検証手段に依存しない抽象化を `tee_type` というキーに集約している．このため，Intel TDX[12] のようにパース難易度の高い多層構造のレポートであっても，対応する検証関数を実装してディスパッチテーブルに登録するだけで対応可能であり，AWS 以外のクラウドへ展開する場合もレポート取得手段の差異は検証関数の内部に閉じる．

また，共通ヘッダによるカプセル化はアプリケーション層で完結するため，下位の通信プロトコルにも依存しない．RA-TLS[8] のように構成証明を通信チャネルと結合する手法では TLS ライブラリの改修を要するのに対し，本手法は

© 2026 Information Processing Society of Japan

TLS 等の既存のセキュアチャネルの上にそのまま載せることができ，本実装の S3 経由の非同期交換のようにチャネル自体の差し替えも容易である．

ただし第 5 章で示したとおり，検証コストは対向の TEE 種別によって決まり，外部の証明書配布サービスへの通信を要する TEE では検証時間が数倍に増大する（Intel TDX も PCK 証明書チェーンを外部取得するため同様の傾向を示すと予想される）．対応 TEE 種別を増やす開発コストは $O(1)$ に抑えられる一方，実行時コストは追加する TEE の検証手段に依存し，かつその差は検証側だけでなくレポート取得側にも現れる点は設計上留意すべきである．検証結果のキャッシュや証明書チェーンの事前取得はこの影響を緩和する手段となりうる．

一方，本手法は組織間で共通ヘッダの形式について合意が形成されていることを前提とする．実運用に向けてはヘッダ形式の標準化，あるいは RFC 9334[1] が定義する Conceptual Message 等の既存の標準表現との対応付けが望ましい．

6.3 リーダープロセス内の初期化コスト

評価 B において，同一リーダープロセス内で 2 件目以降の Nitro Enclaves の検証が 1 件目の約 1/3 の時間で完了する現象を観測した．この短縮は両組織で再現する系統的なものであり，AMD SEV-SNP の構成では観測されなかった．リーダーホスト上で同一のレポートに対する検証を 1 プロセス内で連続実行する測定により，これは継続的な高速化ではなくリーダープロセスあたり 1 回だけ発生する固定コストであることを確認している．

原因を切り分けた結果，この固定コストは実装が用いる CBOR/COSE の処理ライブラリの署名検証パスの初回呼び出しに伴う遅延初期化であることが分かった．これは TEE やハードウェアの性質に由来するものではなく，特定のライブラリ実装に固有の初期化コストであるため，事前にライブラリを実行しておけば解消可能である．

この固定コストはフォロワー台数と無関係にプロセスあたり 1 回のみ発生するため，台数が増えるほど 1 件あたりの平均処理時間は下がる．ただしこれはライブラリの初期化コストが償却されることによる効果であって，提案手法自体の性質を示すものではない．

7. 結論と今後の課題

本稿では，異種 TEE が混在する環境における組織間リモートアテステーションを対象として，リーダーが構成証明レポートに共通ヘッダを付加してカプセル化し，TEE 種別に基づいて検証プロトコルを動的に選択する構成証明代理検証手法を提案した．検証ロジックの抽象化をリーダーに集約することで，開発・保守コストを従来の $O(M \times N)$ から $O(M + N)$ に低減できることを，実装規模に基づく比較により示した．また，AWS 上の実機評価により，異種 TEE 間の相互運用性が得られること，共通ヘッダの処理に伴うオーバーヘッドが処理全体の 0.5 %未満にとどまること，およびフォロワーの構成を変化させても 1 件あたりの処理時間が劣化しないことを確認した．

今後の課題としては，第 3 章で設計を示した nonce 照合処理の実装とその有効性の実験的な確認，Intel TDX を含む 3 種以上の TEE・マルチクラウド環境への拡張，共通ヘッダ形式の標準化と既存の標準表現との対応付け，およびリーダーへの負荷集中・障害への対策が挙げられる．

参考文献

[1] Birkholz, H., Thaler, D., Richardson, M., Smith, N. and Pan, W.: *RFC 9334: Remote ATtestation procedureS (RATS) Architecture* (2023).

[2] Asokan, N., Brasser, F., Ibrahim, A., Sadeghi, A.-R., Schunter, M., Tsudik, G. and Wachsmann, C.: SEDA: Scalable Embedded Device Attestation, *Proc. 22nd ACM SIGSAC Conference on Computer and Communications Security*, pp. 964–975 (2015).

[3] Chen, G., Zhang, Y. and Lai, T.-H.: OPERA: Open Remote Attestation for Intel's Secure Enclaves, *Proc. 2019 ACM SIGSAC Conference on Computer and Communications Security*, pp. 2317–2331 (2019).

[4] Chen, G. and Zhang, Y.: MAGE: Mutual Attestation for a Group of Enclaves without Trusted Third Parties, *31st USENIX Security Symposium (USENIX Security 22)*, pp. 4095–4110 (2022).

[5] Li, P., Li, X. and Fang, L.: DMA: Mutual Attestation Framework for Distributed Enclaves, *International Conference on Information and Communications Security*, Springer Nature Singapore (2024).

[6] Petzi, L., Yahya, A. E. B., Dmitrienko, A., Tsudik, G., Prantl, T. and Kounev, S.: SCRAPS: Scalable Collective Remote Attestation for Pub-Sub IoT Networks with Untrusted Proxy Verifier, *31st USENIX Security Symposium (USENIX Security 22)*, pp. 3485–3501 (2022).

[7] Kwon, H.: Secure and Scalable Device Attestation Protocol with Aggregate Signature, *Symmetry*, Vol. 17, No. 5, p. 698 (2025).

[8] Knauth, T., Steiner, M., Chakrabarti, S., Lei, L., Xing, C. and Vij, M.: Integrating Remote Attestation with Transport Layer Security, arXiv preprint arXiv:1801.05863 (2018).

[9] 千田拓矢，長谷川慶太，永田智大，中川智尋，佐々木一也，寺田雅之：大規模分散システムにおける階層的リモートアテステーション手法の提案と評価，コンピュータセキュリティシンポジウム 2025 論文集，pp. 891–898 (2025).

[10] Amazon Web Services: *AWS Nitro Enclaves User Guide* (online), ⟨https://docs.aws.amazon.com/enclaves/latest/user/nitro-enclave.html⟩ (2026.08.20).

[11] Advanced Micro Devices: *SEV Secure Nested Paging Firmware ABI Specification*, Publication No. 56860 (2025), ⟨https://www.amd.com/content/dam/amd/en/documents/developer/56860.pdf⟩ (2026.08.20).

[12] Intel Corporation: *Intel Trust Domain Extensions (Intel TDX)* (online), ⟨https://www.intel.com/tdx⟩ (2026.08.20).

© 2026 Information Processing Society of Japan